MODELING AND EXPLOITING STOCK MARKET TRENDS WITH HIDDEN MARKOV TECHNOLOGY

JUDE CHELICO

Abstract. In this paper I utilize Hidden Markov Models (HMMs) in order to analyze historical stock market returns and devise a quantitative trading strategy. I begin by introducing topics in pattern recognition and data clustering, building towards a characterization of HMMs. I derive several key estimators and employ optimization techniques to demonstrate the capability of computer vision. Then, having built up the necessary technology to understand the existing literature, I review a body of research which employs HMMs for financial analysis. After establishing the validity of such strategies, I devise, implement, and test my own trading algorithm using historical stock data. I compare the results from my algorithm to important benchmarks such as a basic 'buy and hold' strategy and discuss the algorithm's ability to distinguish regimes. I conclude with this analysis, as well as several notes on potential extensions of this research.

Contents

1. Introduction 2
2. Methods 2
2.1. The k-means algorithm 2
2.2. Gaussian Mixture Models 5
2.3. Hidden Markov Models 13
3. Existing Literature 24
3.1. Quantitative methods in financial modeling 24
3.2. HMMs for stock market prediction 24
4. Online Updating 26
5. Applying HMMs to the Stock Market 29
5.1. The data 29
5.2. A primitive trading algorithm 29
5.3. Online trading algorithm 29
5.4. Backtesting 30
6. Conclusion 33
6.1. Discussion of results 33
6.2. Future opportunities 34

Date: 2026-04-09.

References 35
Appendix A.
Summation Notation of the Log-Likelihood Function 37

1. Introduction

Pattern recognition and prediction is a fundamental issue in financial analysis. We often center our approach on understanding the past in order to create a reasonable expectation for the future. However, the seasonality and cyclicality of financial returns make quantitative analysis more complicated. Additionally, inter- and intraday trends can cause significant losses (or be exploited for significant gain) in the short-term, even as market performance remains moderately strong over time. While propositions like the Efficient Market Hypothesis reject the notion of trends, instead assuming that price fluctuations are random in the absence of relevant information, asset management firms nonetheless achieve market-beating profits from trend-based trading strategies [15]. This creates an incentive to model these trends, both from a financial standpoint as well as an economic curiosity.

One notion of trends in the stock market comes in the idea of regime switching. We can model two regimes: positive 'runs', in which asset values tend to rise, and negative 'reversals', in which asset values tend to fall. One could hypothesize that, in the absence of relevant information such as quarterly earnings or interest rate announcements, the world is continually switching between regimes at random, and price movements are determined by distributions corresponding to each regime rather than a simple random walk. Then, asset price movement remains random, but predictions become steadier. This is because regime changes are weakly less frequent than individual price movements, as the regime can only change at a maximum rate of once per observation. A simple way to model random regime change is Hidden Markov Modeling. HMMs assume an underlying Markov process which governs the sequence of regimes. Each regime has an associated emission distribution, and we observe the emissions in each period. The trick, then, is how we can back out and predict the hidden regime change process from the observable emissions. In this paper, I apply HMMs to financial assets, modeling the hidden states (runs or reversals) using observable asset price fluctuations. I develop a method for forecasting expected variance given the sequence of observation and devise a trading algorithm based on this method.

2. Methods

2.1. The k-means algorithm.

Imagine that you have before you a data set split into rough clusters. Maybe these clusters represent local outbreaks of a

disease, or locations of geological events, or some other randomly distributed set of points that fall into identifiable groups based on their position on a graph. In such a situation, it might be important to cluster the data into rough groups based on the position of each point. This could be done to identify patterns in the data or predict future observations. To the human eye, it's often quite simple to identify these groups. On a two-dimensional graph, you could imagine drawing ellipses that encompass (most of) the points in each group, or starting from some visual measure of center for each group and drawing a border centered on those points. However, these "naked eye" methods suffer from two key errors. That is, they lack both consistency as well as any notion of objectivity. Any two observers are likely to identify slightly different centers, or draw slightly different ellipses, and they would have no definitive way of saying whose work most accurately grouped the data. Additionally, human vision fails in more than three dimensions, as we are unable to properly visually represent the fourth or higher dimensions. We turn to algorithmic computer vision to resolve these issues. One such computer vision algorithm is the k-means algorithm.

2.1.1. *Intuition.* Measures of center are metrics which tell us where the data tends to be located. Such metrics include the simple average and the median. Means are somewhat easier to understand, for both us and machines, and so we will start with the identification of means. One simple and effective method for identifying means in clustered data is k-means clustering. I will explain the algorithm in four steps, following its definition in chapter 9.1 of Bishop [2]. The steps are as follows:

1. Initialize k cluster centers, often randomly distributed around the graph of the data.
2. Identify which data points are closest to each of the generated centers.
3. Set the new location of the generated points to be the mean of the points closest to them.
4. Repeat steps two and three until there is some form of convergence, or until you reach some set limit on the appropriate number of iterations. Typically, convergence is defined as the new locations of the generated points collectively being very close to their prior locations.

This algorithm is quite effective for identifying clusters and their means, as long as the number of hypothesized means (K) is correct. However, there are several ways that k-means clustering can go wrong. An example of successful k-means clustering, as well as two examples of of unsuccessful k-means, is provided in figure 1.

The successful example is displayed on top, showing all clusters properly identified with modeled means (the multicolored icons of varying shape) very close to the true means (the purple dots). The left unsuccessful example shows what happens when the wrong number of means are used. In this

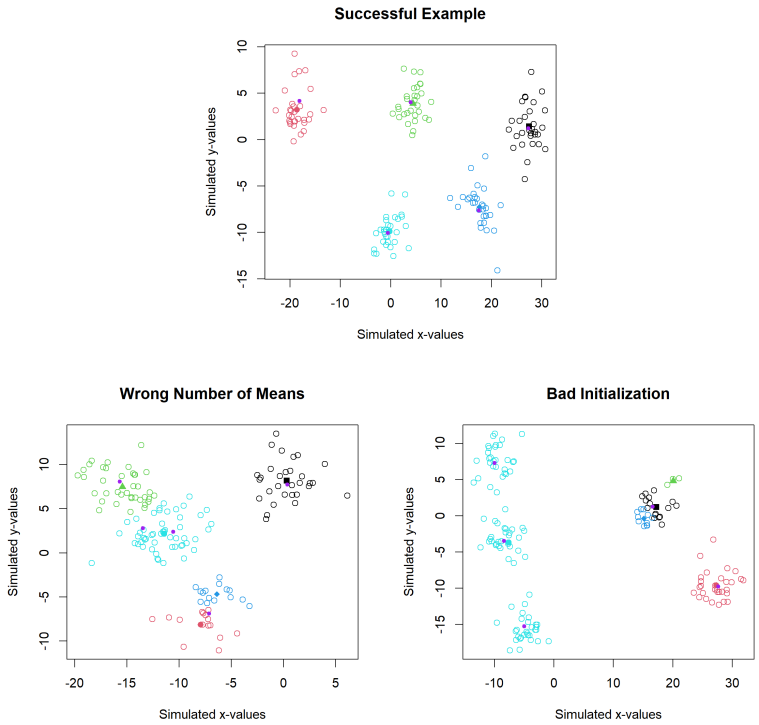

FIGURE 1. Three examples of the k-means algorithm, with varying degrees of success.

example, the data generation function produces one big cluster composed of three smaller clusters, and the algorithm is unable to properly model the means as the clusters overlap. The right unsuccessful example shows what happens when the model means are initialized in bad places, with most of the model means becoming 'trapped' on the righthand side of the graph while the algorithm recognizes three clusters as one. In practice, one might run multiple iterations of k-means clustering in order to find the best fit[1].

2.1.2. *Formalizing k-means.* Now, I will give a more formal treatment to each of the concepts described in the previous section. I will start with the k-means algorithm in d dimensions. The algorithm first initializes K centers $\{\mu_j\}_{j=1}^{K}$ in 'step zero'. My simulations do this by sampling x- and y-coordinates from

[1]This could look like using different numbers of initial centers and/or different initializations in a grid search. Then, a secondary algorithm could evaluate the goodness of fit for each iteration. Such an algorithm might consider the average total distance of all relevant points from each center, addressing cases in which the k-means algorithm predicts a mean in between two data clusters.

a uniform distribution from the minimum to the maximum of the data. Next, in the first step, the algorithm must figure out which $d \times 1$ data points v_i are closest to each $d \times 1$ estimated mean μ_j. My algorithm starts by calculating the square of the Euclidean distance

$$D_{ij} = (v_i - \mu_j)^T (v_i - \mu_j)$$

for each mean, i.e. μ_j for $j \in \{1, k\}$. Then, it reports the mean with the smallest D_{ij} for the current (i^{th}) data point as the mean which that data point is closest to. This can be used to create a vector

$$c = \begin{bmatrix} \operatorname{argmin}_j D_{1j} \\ \operatorname{argmin}_j D_{2j} \\ \vdots \\ \operatorname{argmin}_j D_{Nj} \end{bmatrix}$$

where $\operatorname{argmin}_j D_{ij}$ is defined as the j for which the distance from the data point v_i to the estimated mean μ_j is the smallest, and N is the total number of data points. Once the algorithm has identified the estimated mean closest to each point, it then redefines each estimated mean as the average of each point for which it is the closest mean. Then, the new estimate is

$$\mu_j^{new} = \frac{\sum_{c_i = j} v_i}{N_j}$$

where N_j is the number of points for which $c_i = j$. Finally, the algorithm evaluates the total distance from the new estimated means to the old estimated means

$$D_\mu = \sum_{j=1}^{K} \left(\mu_j^{new} - \mu_j^{old}\right)^T \left(\mu_j^{new} - \mu_j^{old}\right)$$

The algorithm checks whether D_μ is smaller than an arbitrary tolerance. If it is not, then the process repeats from the first step until either D_μ passes the tolerance or a computational limit is reached. Once at least one of those conditions is met, the current estimates μ_j are reported for $j \in \{1, k\}$.

2.2. **Gaussian Mixture Models.** One key weakness of the k-means algorithm is that it gives no impression of the variance of the data, or how it is spread out around its centers. In the example of modeling localized outbreaks of an infectious disease, the variance of the data would provide important information that simple mean identification would not. An outbreak with a small geographical variance could be expected to grow more slowly than one whose patients hailed from across a city. Figure 2 shows an example of two data clusters with drastically different variances, for which k-means would provide no insight into the relative spread of each of these clusters. However, a technique known as Gaussian Mixture Modeling provides a way to model this spread.

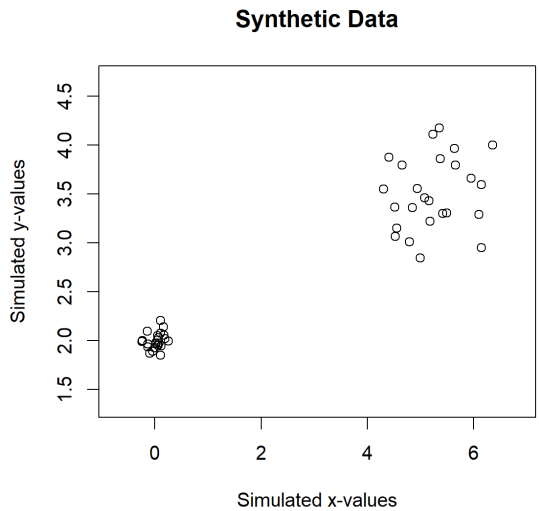

FIGURE 2. Two data clusters with uneven spreads.

2.2.1. *What makes a GMM?.* Gaussian Mixture Models (GMMs) are essentially a generalization of k-means clustering which, rather than modeling each data cluster using a mean and no variance, instead models each cluster as a draw from a multivariate normal distribution [2]. A GMM provides estimates of the means and covariance matrices for each of these predicted distributions. Thus, d-dimensional GMMs are composed of a few key objects, being:

(1) The means $\mu = \{\mu_1, \ldots, \mu_K\}$. Each mean is a $d \times 1$ vector, so μ is $d \times K$.
(2) The covariances $\Sigma = \{\Sigma_1, \ldots, \Sigma_K\}$. Each covariance is a $d \times d$ matrix.
(3) The mixing coefficients $\pi = \{\pi_1, \ldots, \pi_K\}$, a probability distribution over states $j \in \{1, k\}$.

Remember that K is our hypothesized number of clusters. Let j denote each individual cluster, so that μ_j is the mean and Σ_j the covariance matrix of cluster j. Mixing coefficients essentially represent the proportion of the data attributed to each cluster.

These three objects constitute the core framework of a GMM. However, there are other interesting quantities we can evaluate given an existing GMM structure. One of these quantities is the log-likelihood function $\ln p(X|\mu, \Sigma, \pi)$. The log-likelihood function calculates the natural logarithm of the probability of the data occurring as it did assuming a specific set of means, covariances, and mixing coefficients. Maximizing this log-likelihood tells us which combination of these parameters is best at explaining the data we observed.

We use log-likelihood as opposed to normal probabilities primarily to avoid numerical underflow. The probability of any one sequence of states and observations naturally becomes smaller over time due to the extreme number of possible combinations of said states and observations. As this probability approaches zero, rounding error in computation will often lead to it being approximated as zero. This leads to a complete breakdown of our optimization, as we can no longer distinguish between the most or least likely sequences. Thus, we use log-likelihood instead.

2.2.2. *Necessary conditions when fitting a GMM.* The fitting of Gaussian Mixture Models will be given its own section, but first it is important to understand the necessary conditions for maximizing the log-likelihood function. The log-likelihood function is

$$\ln p(X|\mu, \Sigma, \pi) = \sum_{n=1}^{N} \ln \left(\sum_{j=1}^{k} \pi_j \mathcal{N}(x_n|\mu_j, \Sigma_j) \right)$$

where x_n denotes the nth data point (the nth row of the data matrix X) and $\mathcal{N}(x_n|\mu_j, \Sigma_j)$ denotes the multivariate normal density at x_n given the mean μ_j and covariance matrix Σ_j [2, p. 435]. We cannot find a closed-form solution by taking derivatives of this expression, but we can use those derivatives to design an iterative algorithm for maximizing the function [6]. To see this, let's go through the process of taking these derivatives and solving for each of the parameters. We start with the probability density function (PDF) of the multivariate normal distribution:

$$\mathcal{N}(x_n|\mu_j, \Sigma_j) = \frac{1}{(2\pi)^{\frac{d}{2}}} |\Sigma_j|^{-\frac{1}{2}} \exp\left(-\frac{1}{2}(x_n - \mu_j)^T \Sigma_j^{-1} (x_n - \mu_j) \right)$$

Thus, we can take the derivative of the log-likelihood function with respect to the means μ_j and set it equal to 0 to see that, at any point where the log-likelihood function is maximized,

$$0 = -\sum_{n=1}^{N} \frac{\pi_j \mathcal{N}(x_n|\mu_j, \Sigma_j)}{\sum_{i=1}^{K} \pi_i \mathcal{N}(x_n|\mu_i, \Sigma_i)} \left(\Sigma_j^{-1}(x_n - \mu_j) \right)$$

Here, we will pause to observe the meaning of the term

$$\frac{\pi_j \mathcal{N}(x_n|\mu_j, \Sigma_j)}{\sum_{i=1}^{K} \pi_i \mathcal{N}(x_n|\mu_i, \Sigma_i)}$$

Let the $K \times 1$ vector z_n indicate which cluster x_n belongs to, so that one entry z_{nj} of z_n is one (indicating that x_n belongs to state j), and the remaining entries are zero. Suppose we want to calculate the probability that point x_n belongs to the jth cluster

$$p(z_{nj} = 1|x_n)$$

We can use Bayes' rule to perform the following algebra:

$$\begin{aligned} p(z_{nj} = 1|x_n) &= \frac{p(z_{nj} = 1 \cap x_n)}{p(x_n)} \\ &= \frac{p(x_n|z_{nj} = 1)p(z_{nj} = 1)}{p(x_n)} \end{aligned}$$

We have

$$p(x_n|z_{nj} = 1) = \mathcal{N}(x_n|\mu_j, \Sigma_j) \tag{1}$$

$$p(z_{nj} = 1) = \pi_j \tag{2}$$

$$p(x_n) = \sum_{i=1}^{K} \pi_i \mathcal{N}(x_n|\mu_i, \Sigma_i) \tag{3}$$

where (1) can be seen by the fact that we have a distribution for each state, and we are conditioning on the knowledge of which state we are in; (2) is the definition of our prior assumed distribution, and (3) states that $p(x_n)$ is a weighted average of K Gaussians (a marginalization over all possible states from which x_n could have been drawn), which is again inherent to our prior assumptions. Then, we have shown that

$$p(z_{nj} = 1|x_n) = \frac{\pi_j \mathcal{N}(x_n|\mu_j, \Sigma_j)}{\sum_{i=1}^{K} \pi_i \mathcal{N}(x_n|\mu_i, \Sigma_i)}$$

which is precisely the term we singled out above. Therefore, we can say that this term is the (posterior) likelihood that the point x_n belongs to cluster j. We will call this term the responsibility $\gamma(z_{nj})$. Returning to the derivative of the log-likelihood function, we can perform some algebra to see that, when the derivative is 0,

$$\sum_{n=1}^{N} \gamma(z_{nj})\Sigma_j^{-1} x_n = \sum_{n=1}^{N} \gamma(z_{nj})\Sigma_j^{-1} \mu_j$$

We assume that Σ_j^{-1} is nonsingular, i.e. that Σ_j exists, and so we can multiply through by Σ_j and rearrange to get

$$\mu_j = \frac{1}{N_j} \sum_{n=1}^{N} \gamma(z_{nj}) x_n$$

where

$$N_j = \sum_{n=1}^{N} \gamma(z_{nj})$$

can be thought of as the estimated number of points in cluster j. This resembles the calculation of μ_j from the k-means algorithm, as it is essentially

the average of all of the points we expect to be in cluster j, weighted by their likelihood of being in the cluster. A similar process of derivation and algebraic manipulation shown in Bishop [2] yields that

$$\Sigma_j = \frac{1}{N_j} \sum_{n=1}^{N} \gamma(z_{nj})(x_n - \mu_j)(x_n - \mu_j)^T$$

and

$$\pi_j = \frac{N_j}{N}$$

However, as I foreshadowed earlier, these solutions are not closed-form, as the values of each parameter depend on equations that contain not only themselves but also each of the other parameters through their dependence on the responsibilities. Therefore, we cannot solve them for definite values. Despite this, the solutions yielded through derivation and algebra give us methods for estimating the parameters of the underlying distributions given both data and prior estimations. This is exactly where the EM algorithm comes in.

2.2.3. *Fitting a GMM with EM.* In Bishop [2], we also learn that GMMs can be fitted using a process called Expectation-Maximization, or EM. EM is a method of numerically optimizing to find a solution when closed-form solutions are not possible. The general process of the algorithm is somewhat analagous to the k-means algorithm. It begins with guesses for the parameters of the model. Then, it uses probabilistic calculations to guess which data points belong to which cluster based on those parameters. Afterwards, it re-estimates the parameters given the data points that belong to them. This process of evaluating points and guessing parameters repeats until some form of convergence or a preset computational limit is reached.

In the context of GMMs, the EM algorithm relies on several building blocks. They are as follows: the $N \times d$ data matrix X of N observations in d dimensions, means μ_j, covariances Σ_j, mixing coefficients π_j, responsibilities $\gamma(z_{nj})$, and log-likelihood function $\ln p(X|\mu, \Sigma, \pi)$ [2, p. 438]. Remember that each responsibility $\gamma(z_{nj})$ is the likelihood that the nth data point belongs to the jth cluster. It is important to understand the distinction between the mixing coefficients and the responsibilities. It helps to remember that there are k mixing coefficients, one for each cluster, and nk responsibilities, k for each data point.

Given these building blocks, EM proceeds in the framework outlined above. First, guesses for the means, covariances, and mixing coefficients are initialized. Then, in what is called the expectation step, or 'E step', the responsibilities are evaluated for each data point. After that, in what is called the maximization step, or 'M step', the data is used to estimate new values for the parameters, taking into account the responsibilities. This means that the

estimation of the jth cluster will depend more heavily on data points with a higher $\gamma(z_{nj})$, or a higher likelihood of belonging in the jth cluster. Next, the log likelihood is evaluated using the new guesses. Finally, the algorithm repeats the E step, the M step, and evaluation of the log likelihood until its terminal condition is met.

What follows is a more in-depth outline of the EM algorithm. We assume that initial guesses μ, Σ, and π are given and begin with the E-step. In this step, we use our initial guesses to compute the responsibilities $\gamma(z_{nj})$ according to the equation

$$\gamma(z_{nj}) = \frac{\pi_j \mathcal{N}(x_n|\mu_j, \Sigma_j)}{\sum_{i=1}^{K} \pi_i \mathcal{N}(x_n|\mu_i, \Sigma_i)}$$

In other words, we take the weighted likelihood of drawing x_n from cluster j and divide it by the total weighted likelihood of drawing x_n from any of the K clusters. This gives us an assortment of $\gamma(z_{nj})$, where each $\gamma(z_{nj})$ represents our expectation of the likelihood that the nth point is in the jth cluster. This is why this step is called the E-step - it represents an evaluation of expectations given our assumed means, covariances, and mixing coefficients.

Now that the E-step has been completed, we can move on to the M-step. We use γ to calculate new estimates for μ, Σ, and π given the expressions we computed for them in the previous section. That is, we have

$$\begin{aligned}
\mu_j^{new} &= \frac{1}{N_j} \sum_{n=1}^{N} \gamma(z_{nj}) x_n \\
\Sigma_j^{new} &= \frac{1}{N_j} \sum_{n=1}^{N} \gamma(z_{nj})(x_n - \mu_j)(x_n - \mu_j)^T \\
\pi_j^{new} &= \frac{N_j}{N}
\end{aligned}$$

recalling the definition of N_j from section 2.1.2.

Now, we have updated estimates for the model parameters based on the previously calculated responsibilities. These new estimates allow us to return to the E-step and calculate new responsibilities, as new estimates imply new responsibilities. The EM algorithm then repeats until the new estimates are close enough to the old estimates, as determined by some preset tolerance. Once this happens, we have our model, with estimates for the means, covariances, and mixing coefficients. To make the process more clear, I will now present an example.

2.2.4. *A brief example.* In this section, I will generate synthetic data with preset parameters and then fit a GMM to the data, attempting to recover those parameters. The code used for generating and fitting the data was first written by me, then revised and edited using ChatGPT. The key function

in this code takes in a parameter for a seed, which ensures that random outcomes are consistent when run over multiple iterations, and one for the number of clusters to generate. It then uses helper functions to generate data, fit a GMM to that data, and then return both the fitted values and the true data parameters. This allows us to compare the model's prediction of the means and covariance matrices of the multivariate normal patterns which govern the data's distribution with the true underlying distributions. Such a comparison can be seen in figure 3, in which the model's predicted

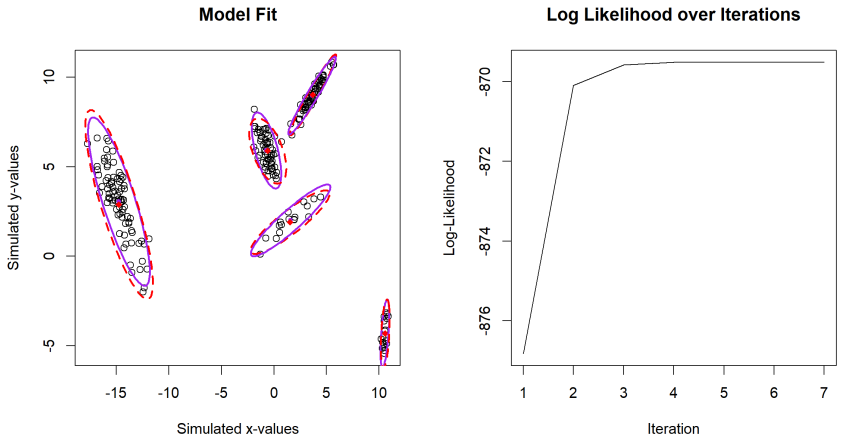

FIGURE 3. An example fit for a GMM and the log-likelihood over 7 iterations.

means are shown by the solid purple dots, predicted variances are shown by the solid purple ellipses, true means are shown by the red diamonds, and true variances are shown by the dotted red ellipses. The figure also displays the value of the log-likelihood function over iterations. The plot of the log-likelihood demonstrates both the increasing and convergent nature of the function as the EM algorithm proceeds. Figures 4, 5, and 6 also compare

Cluster	Predicted mean	Actual mean
1	(-0.71, 5.91)	(-0.59, 5.88)
2	(-14.75, 3.05)	(-14.74, 2.89)
3	(10.54, -4.31)	(10.58, -4.32)
4	(1.58, 2.01)	(1.54, 1.91)
5	(3.62, 8.97)	(3.70, 9.01)

FIGURE 4. Predicted vs. actual means for example GMM

Cluster	Predicted covariance	Actual covariance
1	$\begin{bmatrix} 0.33 & -0.31 \\ -0.31 & 0.75 \end{bmatrix}$	$\begin{bmatrix} 0.52 & -0.28 \\ -0.28 & 0.55 \end{bmatrix}$
2	$\begin{bmatrix} 1.44 & -1.92 \\ -1.92 & 3.69 \end{bmatrix}$	$\begin{bmatrix} 1.72 & -2.36 \\ -2.36 & 4.65 \end{bmatrix}$
3	$\begin{bmatrix} 0.03 & 0.07 \\ 0.07 & 0.60 \end{bmatrix}$	$\begin{bmatrix} 0.03 & 0.05 \\ 0.05 & 0.61 \end{bmatrix}$
4	$\begin{bmatrix} 2.38 & 1.17 \\ 1.17 & 0.67 \end{bmatrix}$	$\begin{bmatrix} 2.35 & 0.99 \\ 0.99 & 0.53 \end{bmatrix}$
5	$\begin{bmatrix} 0.85 & 0.79 \\ 0.79 & 0.79 \end{bmatrix}$	$\begin{bmatrix} 0.81 & 0.80 \\ 0.80 & 0.85 \end{bmatrix}$

FIGURE 5. Predicted vs. actual covariance matrices for example GMM

Cluster	Predicted mixing coefficient	Actual mixing coefficient
1	0.25	0.26
2	0.33	0.33
3	0.08	0.08
4	0.07	0.07
5	0.27	0.27

FIGURE 6. Predicted vs. actual mixing coefficients for example GMM

the predicted means, covariance matrices and mixing coefficients with their true counterparts. Values in the tables are rounded to two decimal places for readability.

The predicted means and covariances are quite close to the true parameters, as can be observed both visually and by examining the values in the tables. The model performed especially well on identifying the mixing coefficients, its predictions being almost exactly correct. Overall, the model performs well. It should be noted that I chose a specific seed to showcase an example of a GMM performing well. This was to avoid needing to perform a grid search across sets of different cluster amounts, tolerances, and other model parameters. In many other cases using different seeds, the clusters overlapped or otherwise appeared in patterns that made the GMM perform poorly and would have necessitated a grid search and/or a more complex initialization process.

2.3. **Hidden Markov Models.** There is often a need to add time and state transition to Gaussian Mixture Models. For example, we may be focused on observing stock return regimes, making it important to model when the market is in both optimistic and pessimistic states. The underlying world state will transition back and forth over time, and we often care about the world state at a given point in time. Thus, both time and state transition become key to our modeling. This section will give a formal treatment to Hidden Markov Models, a framework for adding this sort of transition. The base of an HMM is an unobservable Markov process, a stochastic process which switches between regimes in which the transition probabilities depend only on the current world state, and on no previous world states. In an HMM, the underlying world state determined by the Markov process governs some form of emissions that can be observed. However, the transition matrix and emission distributions themselves are treated as unobservable. An example HMM could be characterized by four objects: a transition matrix

$$A = \begin{bmatrix} 0.95 & 0.035 & 0.015 \\ 0.015 & 0.95 & 0.035 \\ 0.05 & 0.05 & 0.9 \end{bmatrix}$$

and three emission distributions

$$\begin{aligned} E_{Bull} &\sim \mathcal{N}(0.01, 0.025^2) \\ E_{Bear} &\sim \mathcal{N}(-0.01, 0.03^2) \\ E_{Neutral} &\sim \mathcal{N}(0, 0.005^2) \end{aligned}$$

This model describes three 'world states': the bullish, bearish, and neutral states, each having associated distribution parameters. We know that the world has three states as we have three emission functions and a 3×3 transition matrix, meaning that there are 3 states that can be transitioned into and out of. The bullish state samples from a $\mathcal{N}(0.01, 0.025^2)$ distribution, the bearish from a $\mathcal{N}(-0.01, 0.03^2)$ distribution, and the neutral state from a $\mathcal{N}(0, 0.005^2)$ distribution. The first row of A represents the probabilities of transitioning from the bull state to each state in column order of bull, bear, and neutral. That is, when in the bull state, there is a 95% chance to stay in the bull state, a 3.5% chance to transition to the bear state, and a 1.5% chance to transition to the neutral state. The second row of A represents the transition probabilities of the bear state, and the third row represents the transition probabilities of the neutral state. This is an example of a Hidden Markov Gaussian Mixture Model (HMGMM), an HMM that uses Gaussian emmissions. That is, it is a GMM for which an underlying hidden Markov process governs which distribution is being drawn from at any given time. An example of synthetic data drawn from this HMGMM can be found in Figure 7. The figure shows cumulative returns as well as the underlying

FIGURE 7. Synthetic stock returns drawn from an HMGMM, with true underlying regimes highlighted.

market state at any given index. It is important to remember that, when fitting an HMM, we treat the transition matrix and emission distributions as unobservable and try to recover them from the data.

Another way to visualize Hidden Markov GMMs is through a two-dimensional rendition similar to the one found in Figure 3. Observe Figure 8. This figure shows three data clusters, one represented by blue dots, one by green triangles, and one by red squares. The three dotted ellipses represent a one-true standard deviation range about the clusters' true means. The data was drawn separately in these three clusters and then semi-randomized to accurately demonstrate what data drawn from an HMGMM might look like. The data begins at the purple diamond in the lower left of the graph, and each subsequent data point is connected by a black line to the prior data point. Thus, the lines in between the clusters represent state transitions, while lines within the clusters represent times when the model remained in the same state. The data concludes at the purple diamond in the upper left of the graph. This simulated data demonstrates how a Hidden Markov process with Gaussian emissions could create clusters of data, visually representing the true process we are attempting to uncover with our modeling. Compare figure 8 to figure 3. In figure 3, the individual clusters are not connected by an underlying process. Instead, each cluster represents an independent sampling process. Conversely, in figure 8, the clusters are connected by an underlying Markov process, and thus each observation is implicitly connected to the previous observation. This distinction is visualized in figure 9. In figure 9, black arrows represent dependency. That is, the node at the tip of the

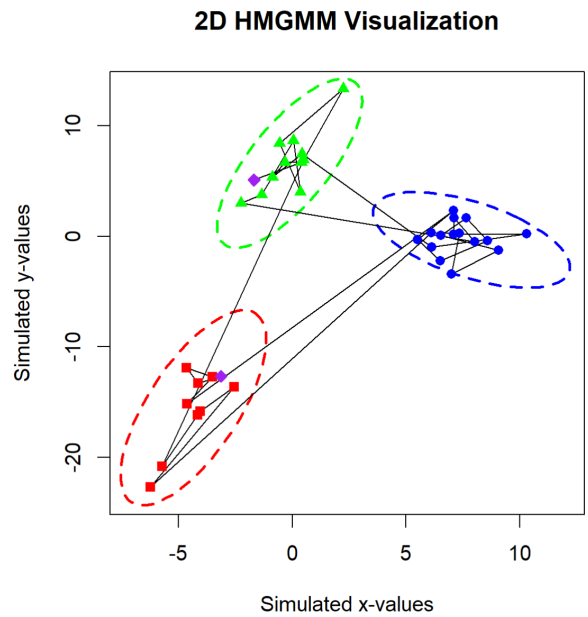

FIGURE 8. Data drawn from a simulated 2D HMGMM with known parameters.

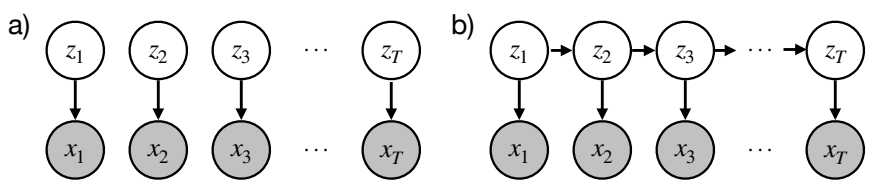

FIGURE 9. Graphical representations for both ordinary GMMs (a) and Hidden Markov GMMs (b).

arrow depends on the node at the base of the arrow. Thus, 9a shows a collection of independent state-draw pairs, where the state at time t z_t influences the data drawn at time t x_t, but not the next state z_{t+1}. On the other hand, 9b shows a model in which both x_t and x_{t+1} depend on z_t.

2.3.1. *Fitting a Hidden Markov GMM.* One key question to my research is how one can back out the parameters of a Hidden Markov Model, particularly one with Gaussian emissions, given a sequence of observable data. That is, if we have a set of sequential data, how can we recover the underlying states those data were drawn from? Additionally, how can we estimate not only the transition matrix for the hidden Markov process, but also the initial state and the emission parameters? Existing literature largely centers around

identifying regimes and patterns in past data (see Maheu et al. [11], for example). I will also be identifying past patterns, but a central concern of my research is how to predict future states and returns given a historical pattern. HMMs provide an ideal framework for both this modeling and this prediction given their ability to switch between world states in a way reminiscent of the real-world transition between market states. Bishop [2] contains a framework for prediction based on HMMs, which I will make heavy use of later. For now, I will be focusing on the methods for fitting an HMGMM to observable data.

2.3.2. *The Forward-Backward Algorithm.* The algorithm I will be using to fit my HMGMMs is the Forward-Backward Algorithm (FBA). The FBA is a form of Expectation-Maximization which makes use of recursion in order to repeatedly estimate and evaluate the parameters of the HMGMM, similarly to how I previously used EM to fit non-HM GMMs. Before outlining the algorithm, it is important to cover some notation. We define θ as the set of model parameters, in this case $\{\pi, A, \phi, K\}$ where π_i is the probability that the model started in state i, A is the transition matrix, ϕ_i is the emission distribution we model for cluster i, and K is the number of states we are modeling. Then, from Equations 13.13 and 13.14 in Bishop [2], we have

$$\gamma(z_n) = p(z_n|X, \theta^{old}) \tag{4}$$

$$\xi(z_{n-1}, z_n) = p(z_{n-1}, z_n|X, \theta^{old}) \tag{5}$$

That is, $\gamma(z_n)$ is the posterior distribution of one latent state variable z_n, while $\xi(z_{n-1}, z_n)$ is the joint posterior distribution of two successive state variables z_{n-1}, z_n. Then, for each n, $\gamma(z_n)$ is a K-long vector of nonnegative numbers that sum to 1, and $\xi(z_{n-1}, z_n)$ is a $K \times K$ matrix of nonnegative numbers that also sum to 1 in total. This probability is conditioned on both the data X and the assumed parameters θ^{old}, meaning that $\gamma(z_{nk})$ denotes the conditional probability that $z_{nk} = 1$, or that the nth data point is drawn from the kth regime given the rest of the data and the assumed parameters. This definition of $\gamma(z_n)$ is simply a generalized expression, with the same interpretation as the GMM case. We condition on the data due to the regime-switching nature of the model. If the data points surrounding a given observation are drawn from a certain regime, it makes it more likely that that observation comes from the same regime. The parameters are important as they give some measure of what the regimes could be expected to look like.

Additionally, we have that $\xi(z_{n-1,j}, z_{nk})$ denotes the conditional probability that the $n-1^{\text{th}}$ point is drawn from state j, while the nth point is drawn from state k. This probability is also conditioned on the data and the assumed parameters, which are relevant for similar reasons. One thing to note is that the assumed parameters θ^{old} include the transition matrix A, which is quite relevant to the construction of $\xi(z_{n-1,j}, z_{nk})$.

2.3.3. *Bounding The Log-Likelihood.* In the forward-backward version of the EM algorithm, we do not directly optimize $p(X|\theta)$, as computing this likelihood becomes extremely computationally intensive quite quickly [8]. Instead, we seek to construct a lower bound on the log-likelihood $\ln p(X|\theta)$ and then optimize that lower bound. We do this for the aforementioned benefit to numerical stability.

To construct a lower bound on $\ln p(X|\theta)$, we perform some algebra and use Jensen's inequality to note that

$$\begin{aligned}
\ln p(X|\theta) &= \ln \sum_Z p(X, Z|\theta) \\
&= \ln \sum_Z q(Z) \frac{p(X, Z|\theta)}{q(Z)} \\
&= \ln \mathbb{E}_{q(Z)} \left[\frac{p(X, Z|\theta)}{q(Z)} \right] \\
&\geq \mathbb{E}_{q(Z)} \left[\ln \frac{p(X, Z|\theta)}{q(Z)} \right]
\end{aligned}$$

where $q(Z)$ is some density on Z. The fourth line above uses Jensen's inequality, as the natural logarithm is concave. Now, we can rearrange this to say that

$$\ln p(X|\theta) \geq \mathbb{E}_{q(z)} \left[\ln p(X, Z|\theta) \right] - \mathbb{E}_{q(z)} \left[\ln q(Z) \right]$$

We call the first term on the righthand side the Evidence Lower Bound, or the ELBO, and the second term the entropy of q. Jensen's inequality holds with equality here when $\ln \frac{p(X,Z|\theta)}{q(Z)}$ is constant. Thus, to construct a tight lower bound (one such that no greater quantity is also a lower bound), we want to pick $q(Z)$ such that it is proportional to $p(X, Z|\theta)$, keeping in mind that it must be a density over Z. We can choose

$$q(Z) = \frac{p(X, Z|\theta)}{\sum_{Z'} p(X, Z'|\theta)} = \frac{p(X, Z|\theta)}{p(X|\theta)} = p(Z|X, \theta)$$

a posterior density over states $j \in \{1, k\}$. Then,

$$\begin{aligned}
\ln p(X|\theta) &= \mathbb{E}_{p(Z|X,\theta^{old})} \left[\ln p(X, Z|\theta) \right] - \mathbb{E}_{p(Z|X,\theta^{old})} \left[\ln p(Z|X, \theta) \right] \\
&= Q(\theta|\theta^{old}) + H(\theta|\theta^{old})
\end{aligned}$$

We now seek to prove that the log-likelihood is increasing when $Q(\theta|\theta^{old})$ is increasing. To do so, we must show that $H(\theta|\theta^{old}) \geq H(\theta^{old}|\theta^{old})$ for new

model parameters θ. Observe that

$$\begin{aligned} H(\theta|\theta^{old}) - H(\theta^{old}|\theta^{old}) &= \mathbb{E}_{p(Z|X,\theta^{old})}\left[\ln \frac{p(Z|X,\theta)}{p(Z|X,\theta^{old})}\right] \\ &= \sum_Z p(Z|X,\theta^{old}) \ln \frac{p(Z|X,\theta)}{p(Z|X,\theta^{old})} \end{aligned}$$

but this is the Kullback-Liebler divergence of the two distributions, and so it must be greater than or equal to zero. Then,

$$\begin{aligned} \ln p(X|\theta) - \ln p(X|\theta^{old}) &= Q(\theta|\theta^{old}) + H(\theta|\theta^{old}) - \Big(Q(\theta^{old}|\theta^{old}) + H(\theta^{old}|\theta^{old})\Big) \\ &\geq Q(\theta|\theta^{old}) - Q(\theta^{old}|\theta^{old}) \end{aligned}$$

Then, given this inequality along with the fact that $Q(\theta|\theta^{old})$ is a tight lower bound, we know that increasing $Q(\theta|\theta^{old})$ must also increase $\ln p(X|\theta)$ by the same amount. That is, we know that $Q(\theta|\theta^{old})$ is either equal to the log-likelihood or infinitesimally close to it, so increasing $Q(\theta|\theta^{old})$ forces the log-likelihood up as well.

To recap, we have

$$\begin{aligned} Q(\theta|\theta^{old}) &= \mathbb{E}_{p(Z|X,\theta^{old})}\left[\ln p(X,Z|\theta)\right] \\ &= \mathbb{E}\left[\ln p(z_1)\right] + \sum_{n=2}^{N} \mathbb{E}\left[\ln p(z_n|z_{n-1})\right] + \sum_{n=1}^{N} \mathbb{E}\left[\ln p(x_n|z_n)\right] \end{aligned}$$

Relating this back to the transition matrix, notice that

$$\ln p(z_n|z_{n-1}) = \sum_{j=1}^{K}\sum_{k=1}^{K} z_{n-1,j} z_{nk} \ln A_{jk}$$

where A_{jk} is the kth entry in the jth row of the transition matrix, and the product $z_{n-1,j} z_{nk}$ is 1 only when we transition from state j to state k, which only occurs once in the double sum. Then, by the linearity of expectation, we have

$$\begin{aligned} \sum_{n=2}^{N} \mathbb{E}\left[\ln p(z_n|z_{n-1})\right] &= \sum_{n=2}^{N} \mathbb{E}\left[\sum_{j=1}^{K}\sum_{k=1}^{K} z_{n-1,j} z_{nk} \ln A_{jk}\right] \\ &= \sum_{n=2}^{N}\sum_{j=1}^{K}\sum_{k=1}^{K} \underbrace{\mathbb{E}\left[z_{n-1,j} z_{nk}\right]}_{\xi(z_{n-1,j}, z_{nk})} \ln A_{jk} \end{aligned}$$

Repeating a similar process with the other terms [8], we get

$$Q(\theta|\theta^{old}) = \sum_{k=1}^{K} \gamma(z_{1k}) \ln \pi_k + \sum_{n=2}^{N} \sum_{j=1}^{K} \sum_{k=1}^{K} \xi(z_{n-1,j} z_{nk}) \ln A_{jk} + \sum_{n=1}^{N} \sum_{k=1}^{k} \gamma(z_{nk}) \ln p(x_n|\phi_k)$$

See appendix A for more detail.

2.3.4. *Deriving The Optimal Parameters.* We are now ready to derive μ_k and Σ_k by optimizing the likelihood function in the M-step. This optimization maximizes the likelihood that of the data being drawn from these parameters $\phi_k = \{\mu_k, \Sigma_k\}$. Because we are optimizing with respect to the ϕ_k, we only care about the term where they appear:

$$\sum_{n=1}^{N} \sum_{k=1}^{K} \gamma(z_{nk}) \ln p(x_n|\mu_k, \Sigma_k)$$

We can derive this term with respect to μ_k. Since we are optimizing one state at a time, we can remove the inner sum to get

$$\begin{aligned}
\frac{\partial}{\partial \mu_k} \sum_{n=1}^{N} \sum_{k=1}^{K} \gamma(z_{nk}) \ln p(x_n|\phi_k) &= \sum_{n=1}^{N} \frac{\partial}{\partial \mu_k} \left[\gamma(z_{nk}) \ln p(x_n|\phi_k)\right] \\
&= \sum_{n=1}^{N} \gamma(z_{nk}) \frac{\partial}{\partial \mu_k} \left[\ln \left[(2\pi)^{-\frac{k}{2}} |\Sigma_k|^{-\frac{1}{2}} \exp\left(-\frac{1}{2}(x_n - \mu_k)^T \Sigma_k^{-1} (x_n - \mu_k)\right)\right]\right] \\
&= \sum_{n=1}^{n} \gamma(z_{nk}) \frac{\partial}{\partial \mu_k} \left[\ln \left(\exp\left(-\frac{1}{2}(x_n - \mu_k)^T \Sigma_k^{-1} (x_n - \mu_k)\right)\right)\right] \\
&= \sum_{n=1}^{N} \gamma(z_{nk}) \Sigma_k^{-1} (x_n - \mu_k)
\end{aligned}$$

Setting this derivative equal to 0 and solving, we get

$$\mu_k = \frac{\sum_{n=1}^{N} \gamma(z_{nk}) x_n}{\sum_{n=1}^{N} \gamma(z_{nk})} \tag{6}$$

Appendix A3 in Gundersen [8] also derives the optimal Σ_k, notating $\gamma(z_{nk})$ as $\mathbb{E}[z_{nk}]$. Using those results, we have

$$\Sigma_k = \frac{\sum_{n=1}^{N} \gamma(z_{nk})(x_n - \mu_k)^T (x_n - \mu_k)}{\sum_{n=1}^{N} \gamma(z_{nk})} \tag{7}$$

Pause for a moment to reflect on these values. The numerator of μ_k is the sum of all N data points weighted by the responsibilities, the likelihood that each data point comes from the given hidden state k. This gives us an average value for points in that distribution conditioned on the likelihood that they are drawn from it. The denominator is simply a scaling constant. It is often normalized to 1, but it remains in my notation for the purpose of generalization. Σ_k is a bit more complicated, but it helps to compare it to the definition of one-dimensional standard deviation. Essentially, Σ_k is the sum of squared deviations from μ_k, again weighted by the responsibilities and scaled by its denominator.

Now, we will move to the mixing coefficients and transition matrix. Appendix A shows that the likelihood function

$$\begin{aligned} Q(\theta, \theta^{old}) &= \sum_Z p(Z|X, \theta^{old}) \ln p(X, Z|\theta) \\ &= \sum_{k=1}^{K} \gamma(z_{1k}) \ln \pi_k + \sum_{n=2}^{N} \sum_{j=1}^{K} \sum_{k=1}^{K} \xi(z_{n-1,j}, z_{nk}) \ln A_{jk} \\ &+ \sum_{m=1}^{N} \sum_{k=1}^{K} \gamma(z_{mk}) \ln p(x_m|\phi_k) \end{aligned}$$

Maximizing this function with respect to π and A, we have equations (13.18) and (13.19) in Bishop [2]:

$$\begin{aligned} \pi_k &= \frac{\gamma(z_{1k})}{\sum_{j=1}^{K} \gamma(z_{1j})} \\ &= \gamma(z_{1k}) \end{aligned} \tag{8}$$

$$A_{jk} = \frac{\sum_{n=2}^{N} \xi(z_{n-1,j}, z_{nk})}{\sum_{l=1}^{K} \sum_{n=2}^{N} \xi(z_{n-1,j}, z_{nl})} \tag{9}$$

These expressions make intuitive sense. Here, π_k simply tells us that the mixing coefficient we assign to state k, or the responsibility it has for the point x_1, is $\gamma(z_{1k})$, the probability of x_1 being drawn from distribution k. Additionally, A_{jk} tells us that the likelihood of transitioning from state j to

state k is derived from the likelihood that such transitions occurred in the data.

We now have a collection of relatively clean expressions for the parameters of our model. However, we still need a way to obtain the responsibilities and ξ-values. It is here where we utilize the Forward-Backward Algorithm. Equations 13.33 and 13.43 in Bishop [2] use Bayes' theorem, conditional independence and the product rule to derive that

$$\begin{aligned}\gamma(z_n) &= p(z_n|X) \\ &= \frac{p(X|z_n)p(z_n)}{p(X)} \\ &= \frac{p(x_1,\ldots,x_n|z_n)p(x_{n+1},\ldots,x_N|z_n)p(z_n)}{p(X)} \\ &= \frac{p(x_1,\ldots,x_n,z_n)p(x_{n+1},\ldots,x_N|z_n)}{p(X)} \\ &= \frac{\alpha(z_n)\beta(z_n)}{p(X)}\end{aligned} \tag{10}$$

where $p(X) = p(X|\theta^{old})$ represents the likelihood function and

$$\alpha(z_n) \equiv p(x_1,\ldots,x_n,z_n) \tag{11}$$

$$\beta(z_n) \equiv p(x_{n+1},\ldots,x_N|z_n) \tag{12}$$

That is, $\alpha(z_n)$ is the joint probability of observing all of the given data up to time n and the state z_n (the "forward message"), while $\beta(z_n)$ is the conditional probability of all future data until time N given the value of z_n (the "backward message") [2]. These definitions for α and β will help simplify future expressions. We can similarly express $\xi(z_{n-1}, z_n)$ in terms of these values:

$$\begin{aligned}\xi(z_{n-1},z_n) &= p(z_{n-1},z_n|X) \\ &= \frac{p(X|z_{n-1},z_n)p(z_{n-1},z_n)}{p(X)} \\ &= \frac{p(x_1,\ldots,x_{n-1}|z_{n-1})p(x_n|z_n)p(x_{n+1},\ldots,x_N|z_n)p(z_n|z_{n-1})p(z_{n-1})}{p(X)} \\ &= \frac{\alpha(z_{n-1})p(x_n|z_n)p(z_n|z_{n-1})\beta(z_n)}{p(X)}\end{aligned} \tag{13}$$

Note that 13 also uses conditional independence property (13.29) from Bishop [2] in the third step.

Both $\alpha(z_n)$ and $\beta(z_n)$ can be defined recursively. To see this for $\alpha(z_n)$, note that

$$\begin{aligned}
\alpha(z_n) &= p(x_1, \ldots, x_n, z_n) \\
&= p(x_1, \ldots, x_n | z_n) p(z_n) \\
&= p(x_n | z_n) p(x_1, \ldots, x_{n-1} | z_n) p(z_n) \\
&= p(x_n | z_n) p(x_1, \ldots, x_{n-1}, z_n) \\
&= p(x_n | z_n) \sum_{z_{n-1}} p(x_1, \ldots, x_{n-1}, z_{n-1}, z_n) \\
&= p(x_n | z_n) \sum_{z_{n-1}} p(x_1, \ldots, x_{n-1}, z_n | z_{n-1}) p(z_{n-1}) \\
&= p(x_n | z_n) \sum_{z_{n-1}} p(x_1, \ldots, x_{n-1} | z_{n-1}) p(z_n | z_{n-1}) p(z_{n-1}) \\
&= p(x_n | z_n) \sum_{z_{n-1}} p(x_1, \ldots, x_{n-1}, z_{n-1}) p(z_n | z_{n-1}) \\
&= p(x_n | z_n) \sum_{z_{n-1}} \alpha(z_{n-1}) p(z_n | z_{n-1}) \qquad (13)
\end{aligned}$$

where we make use of the definition of $\alpha(z_n)$, the product rule of probability, the independence of x_n given z_n, the product rule again, the sum rule of probability, the product rule once again, the independence of z_n given z_{n-1}, the product rule one last time, and finally equation (11). This derivation follows one done on page 620 of Bishop [2]. Equation (13.37) in Bishop [2] also tells us the initial condition for the recursion

$$\alpha(z_1) = p(x_1, z_1) = \prod_{k=1}^{K} (\pi_k p(x_1 | \phi_k))^{z_{1k}} \qquad (14)$$

That is, the initial condition is given by the product of the assumed likelihood of the initial state with the likelihood of the first data point given the assumed parameters of said state. The exponent z_{1k} is a binary value, either 0 or 1, and filters out all values in the product except for the one corresponding to the initial state of the model.

A similar derivation can be performed to show that

$$\beta(z_n) = \sum_{z_{n+1}} \beta(z_{n+1}) p(x_{n+1} | z_{n+1}) p(z_{n+1} | z_n) \qquad (15)$$

and, using $n = N$ in $\gamma(z_n)$ while replacing $\alpha(z_N)$ with its definition,

$$p(z_N | X) = \frac{p(z_N, X) \beta(z_N)}{p(X)} \iff \beta(z_N) = 1$$

Given the definition of conditional probability. Thus, we set $\beta(z_N) = 1$ for all possible settings of z_N.

The general process of EM with the Forward-Backward Algorithm is, as described in Bishop [2]:

(1) Initialize guesses for the parameters of the HMGMM (the mixing coefficients π, transition matrix A, means μ and covariances Σ.
(2) Run the forward recursion to define $\alpha(z_n)$ and the backward recursion to define $\beta(z_n)$.
(3) Use these parameters and equations 10 and 13 to evaluate $\gamma(z_n)$ and $\xi(z_{n-1}, z_n)$. Also evaluate the likelihood function.
(4) Use the new $\gamma(z_n)$ and $\xi(z_{n-1}, z_n)$ to calculate new parameter estimates for the HMGMM through equations 8, 9, 6, and 7.
(5) Repeat steps 2-4 until some convergence criterion is met, for example a lack of significant change in the likelihood function.

The following section will walk through an example application of this algorithm.

2.3.5. *An example fit.* Recall the synthetic stock return data displayed in Figure 7. This data was generated using a Markov process with Gaussian emissions. The transition matrix was as follows:

$$A = \begin{bmatrix} 0.95 & 0.035 & 0.015 \\ 0.015 & 0.95 & 0.035 \\ 0.05 & 0.05 & 0.9 \end{bmatrix}$$

Additionally, there were three underlying states: a bullish state which followed a $\mathcal{N}(0.01, 0.025^2)$ distribution, a bearish state with a $\mathcal{N}(-0.01, 0.03^2)$ distribution, and a neutral state with a $\mathcal{N}(0, 0.005^2)$ distribution.

Using the R package depmixS4 [13] to run the Forward-Backward Algorithm on the 1,000 observations generated from this process results in the estimates

$$\hat{\pi} = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}, \qquad \hat{A} = \begin{bmatrix} 0.920 & 0.040 & 0.041 \\ 0.008 & 0.972 & 0.020 \\ 0.035 & 0.012 & 0.953 \end{bmatrix}$$

$$\hat{\mu} = \begin{bmatrix} 0.000 \\ 0.011 \\ -0.006 \end{bmatrix}, \qquad \hat{\sigma} = \begin{bmatrix} 0.005 \\ 0.024 \\ 0.030 \end{bmatrix}$$

after 109 iterations. Note that, presumably due to some rounding error, the first row of $\hat{A}$ does not meet the summation constraints - it should sum to 1. These estimates indicate that the model has identified state 1 as the neutral state, state 2 as the bullish state, and state 3 as the bearish state. It correctly assumes with complete certainty that the model began in the neutral state. It also almost exactly identifies the means and standard deviations of each

regime, with the furthest estimate being the estimate for the mean of the bearish state, being 0.004 away. In the following sections, I will expand into the realm of inference using HMMs and apply the FBA to real-world stock data.

3. Existing Literature

3.1. **Quantitative methods in financial modeling.** In the period between the Great Recession in 2008 and the year 2019, financial modeling techniques have evolved significantly. Traditional methods such as the Efficient Market Hypothesis and the Capital Asset Pricing Model fail to properly explain phenomena such as irrational trends and panic-induced crashes. In their place, new models have arisen which attempt to explain these phenomena, along with other anomalies presented by empirical analysis of financial time series [14]. Such models avoid transformations which lead to the loss of important memory data and attempt to account for trends. A summary of such models, largely based around traditional time series methods like the autoregressive (AR) and moving average (MA) models, can be found in appendix A2.3 of Vogl [14]. Additionally, Vogl discusses Markov regime switching models like the HMGMM methods I use in this paper. Vogl finds that singular models are, in the aggregate, quite limited when it comes to modeling and forecasting financial markets, and that more advanced strategies such as model families are more well-suited to financial data. Despite this, there is some benefit to be seen when modeling financial data with Markov regime switching.

3.2. **HMMs for stock market prediction.** Hidden Markov Models are not always favored by quantitative traders, as they are generally viewed as being more suited to backward-looking regime identification than forecasting. However, there does exist a body of research which utilizes HMMs for stock prediction, and even some which devise trading strategies based on the technology. Of particular note is the work done by Shaul Wisebourt in his 2011 Masters thesis [15]. In this thesis, Wisebourt employs a variant of HMMs known as Hierarchical HMMs. An exact characterization of HHMMs is beyond the scope of this thesis, but I will provide a brief example. Imagine a world in which there are two 'long run' states, either a bullish or bearish state. However, within these two long run states, there are short-term 'trends', either a run or a reversal. For simplicity, we will assume that both long run states are composed of runs and reversals, but this is not necessary to the structure of an HHMM. Neither the long run state nor the short-term trends are directly observable, but we could observe emissions from the model

and use those emissions to build an HHMM much in the same way that we might build a standard HMM[2].

Using this technology, along with a data cleaning and formatting tool referred to as the Limit Order Book analyzer, Wisebourt develops a simple trading strategy in [15]. Wisebourt's algorithm buys when it predicts a change into the run state and sells when it predicts a change into the reversal state. Wisebourt [15] is particularly motivated by the contradiction between the Efficient Market Hypothesis and the meteoric success of some asset management firms. The EMH assumes that, in markets where relevant information spreads rapidly, stock prices move logically and straightforwardly, and so traders cannot profit off of irrational trends or arbitrary price movements. However, firms like hedge funds have seen immense, market-beating profits by exploiting market trends, among other tactics. The economic implications of a successful quantitative trading algorithm will not be discussed extensively in this paper, but it remains an interesting framing to motivate the study.

Wisebourt's algorithm [15] relies on Viterbi optimization to generate a sequence of likely states based on the data, optimizing

$$\{z_1, \ldots, z_n\} = \arg \max_{z_1,\ldots,z_n} \{p(z_1, \ldots, z_n | x_1, \ldots, x_n)\}$$

Then, if the model finds itself exiting a run or entering a reversal, it will exit the market by selling a given security. However, if it is entering a run or exiting a reversal, it will enter the market by buying that security. This algorithm is rather simple, but it proves relatively effective in high-liquidity environments. Here, 'liquidity' means that a stock is traded frequently, and thus easily convertible into cash, or 'liquidated'. Figure 21 in Wisebourt [15] displays this fact, as well as the relatively poor performance of the algorithm in bottom-half liquidity quartiles. Wisebourt's results suggest that there does exist some trend in markets that can be exploited using HMM techniques, validating the pursuit of an effective quantitative trading algorithm using HMMs.

Given that my paper utilizes regular Hidden Markov Modeling, rather than its Hierarchical form, we may want to modify our forecasting approach to better match our methods. This is because we are seeking to exploit short-term market volatility, which is subject to frequent change, and not long-term bull or bear markets (runs or reversals) whose trends are spread out over weeks or months. Thus, by attempting to compute the next market state or expected returns in the next state, we are effectively asking the model whether it expects the asset to increase in value on average. For highly-traded assets like ETFs that track the S&P 500, the answer to this question

[2]For a more in-depth, but still concise, explanation, consider Figures 13 and 14 in Wisebourt [15].

is nearly always yes. This causes the algorithm to devolve into a buy-and-hold strategy, as it is overly optimistic. For this paper, we must find another method. The literature offers a solution in the form of variance minimization. Baker et al. [1] discusses the 'Low Volatility Anomaly', or the counterintuitive result that assets with a lower variance historically outperform those with a higher variance. This result goes against the widely accepted idea of a 'risk premium', the concept that investors might demand a higher expected return from a more volatile asset. Instead, in Baker et al. [1], the authors make a note of the fact that, "[o]ver the past 41 years, high volatility and high beta stocks have substantially underperformed low volatility and low beta stocks in U.S. markets". Here, 'beta' refers to the volatility of an asset's value relative to the broader market. Thus, the world explained by Baker et al. [1] is one in which we might be able to increase returns by only participating in the market when our model predicts a low variance in the next state.

4. Online Updating

One issue that arises when using Hidden Markov Models for prediction is that they do not excel at forecasting multiple periods into the future, and are more well-suited to short-term prediction, such as of the next state or observation. This is problematic when attempting to devise an effective trading algorithm based on HMM prediction. One option would be to employ a 'moving fit' by fitting a new model on a predetermined window of returns, for example the previous 100 periods of returns, before predicting the next period's outcome. Then, after the period's outcome was predicted, acted upon, and then observed in its true value, the process would be repeated in the next period. However, this method is not ideal for several reasons. First of all, it is extremely computationally intensive, and thus rather slow, making it unsuitable for high-frequency trading. Secondly, it is prone to overfitting, and will likely perform wildly differently depending on market conditions including overall trend and liquidity. A strong alternative is presented in the form of online updating HMMs.

Online updating is a method for fitting HMMs in which the model constantly adapts and changes its parameters in response to new data. This allows us to utilize a dynamic model which can continue to predict new periods, without needing to fit an entirely new model in every period. In [3], Oliver Cappé lays out an algorithm for fitting and updating an HMGMM. Cappé's algorithm relies on the use of some quantities that we are familiar with, such as the responsibilities $\gamma(z_{nk})$, and some new values. These values include the 'sufficient statistics' $S_n^q(i,j)$ and $S_{n,d}^g(i)$, as well as the intermediate quantities $\rho_n^q(i,j,k)$, $\rho_{n,d}^g(i,k)$, $g_{\theta_n}(k,x_n)$, and $r_n(i|j)$. Essentially, the intermediate quantities are calculated from the current parameters and the

data, and the sufficient statistics are calculated from the intermediate quantities, then used to calculate the new parameters. The basic outline of the algorithm is as follows:

(1) Initialize the model parameters θ_0, the responsibilities $\gamma(z_{0k})$, and the intermediate quantities $\rho_0^q(i,j,k)$, and $\rho_{0,d}^g(i,k)$.
(2) Calculate $g_{\theta_0}(k, x_0)$ and use it to update the responsibilities.
(3) E-step: update ρ^q and ρ^g using the new r_n.
(4) M-step: update the sufficient statistics and then the model parameters, including the distributions and transition matrix.
(5) Repeat steps 2-4 for as long as you want (and/or have data).

Now, I will lay out the steps in more detail. First of all, define $\delta(i)$ to be the Kronecker delta, so that it is 1 if and only if $i = 0$. Also, let (b_n) be a weakly decreasing sequence of numbers within $(0, 1)$[3]. Finally, recall that we defined the transition matrix A, and A_{ij} denotes the entry of A in the ith row and the jth column. Now, we are ready to perform the initialization and updates. Cappé [3] defines

$$g_{\theta_n}(k, x_n) = \exp\left(-\frac{(x_n - \mu_n(k))^2}{2\sigma_k}\right)$$
$$r_{n+1}(i|j) = \frac{\gamma(z_{ni})A_{ij}}{\sum_{k=1}^{K}\gamma(z_{nk})A_{kj}}$$

That is, $g_{\theta_n}(k, x_n)$ is the probability density of x_n at state k, while $r_{n+1}(i|j)$ are the backward-looking probabilities $p(z_{ni}|z_{n+1,j}, X_{0:n})$. Then, Cappé [3] initializes

$$\gamma(z_{0k}) = \frac{p(x_0|\theta_0, z_0 = k)g_{\theta_0}(k, x_0)}{\sum_{i=1}^{K} g_{\theta_0}(k, x_0)}$$

where $p(x_0|\theta_0, z_0 = k)$ is calculated given some assumed prior distribution for $X|\theta, Z$. In my implementation, this distribution will be determined by an initial HMM fit as a foundation for the online updates. Then, the core updating loop begins. In this loop, we first update the responsibilities such that

$$\gamma(z_{n+1,k}) = \frac{\sum_{i=1}^{K}\gamma(z_{ni})A_{ik}g_{\theta_n}(k, x_{n+1})}{\sum_{j=1}^{K}\sum_{l=1}^{K}\gamma(z_{nj})A_{jl}g_{\theta_n}(l, x_{n+1})} \tag{16}$$

[3]In order to guarantee convergence to an optimum, (b_n) theoretically must diverge to infinity, while its square must converge to a value less than infinity. I work with constant step sizes in my algorithm to avoid degeneracy in the variance while sacrificing the guarantee of converging exactly to the optimum. See Lakshminarayanan et al.[9] for more detail.

Then, in the E-step, we calculate

$$\rho^q_{n+1}(i,j,k) = b_{n+1}\delta(j-k)r_{n+1}(i|j) + (1-b_{n+1})\sum_{l=1}^{K}\rho^q_n(i,j,l)r_{n+1}(l|k) \tag{17}$$

a convex combination of the new and old responsibilities, and

$$\rho^g_{n+1,d}(i,k) = b_{n+1}\delta(j-k)x^d_{n+1} + (1-b_{n+1})\sum_{l=1}^{K}\rho^g_{n,d}(i,l)r_{n+1}(l|k) \tag{18}$$

for $d = 0, 1, 2$, recalling our earlier definition of $r_{n+1}(i|j)$. Finally, in the M-step, we calculate

$$S^q_{n+1}(i,j) = \sum_{l=1}^{K}\rho^q_{n+1}(i,j,l)\gamma(z_{n+1,l})$$

$$A_{ij} = \frac{S^q_{n+1}(i,j)}{\sum_{l=1}^{K} S^q_{n+1}(i,l)}$$

$$S^g_{n+1,d}(i) = \sum_{l=1}^{K}\rho^g_{n+1,d}(i,l)\gamma(z_{n+1,l})$$

$$\mu_{n+1,k} = \frac{S^g_{n+1,1}(k)}{S^g_{n+1,0}(k)} \tag{19}$$

$$\sigma^2_{n+1,k} = \frac{S^g_{n+1,2}(k) - \mu^2_{n+1,k}S^g_{n+1,0}(k)}{S^g_{n+1,0}(k)} \tag{20}$$

where $\mu_{n+1,k}$ and $\sigma^2_{n+1,k}$ denote the mean and variance of state k at the data point $n+1$, respectively. Take a moment to intuitively understand these expressions for the predicted means and standard deviations, and to compare them from the ones we previously derived from EM. In μ_k, for example, $S^g_{n+1,1}(k)$ is quite similar to the numerator $\sum_{n=1}^{N}\gamma(z_{nk})x_n$ in equation 6. This is because it is fundamentally some combination of the x_n values weighted by the responsibilities. We can draw similar parallels between the other components of the fractions which make up $\sigma^2_{n+1,k}$. Thus, we can see that online updating yields results in line with the previous method of EM, while maintaining the capability to respond to new data.

The ability to continuously update an existing model, rather than fitting a new model for each observation, allows for extremely smooth and efficient modeling using HMGMMs. In the following section, this will enable me to design an effective trading algorithm based on HMGMM technology.

5. Applying HMMs to the Stock Market

5.1. **The data.** The primary data I will use to train my algorithm and test its performance will be historical data on the returns of the S&P 500, an index fund which tracks the value of the 500 top-performing stocks in the market. This data is ideal for my applications, as the S&P 500 is highly liquid, a condition important to the performance of HMMs, as outlined in Wisebourt [15]. Additionally, the S&P 500 is widely considered to be a strong indicator of overall market performance, so training and testing on it should prepare the algorithm for standard market conditions. The data I use catalogs over 20 years of market returns over 8199 days, and was acquired from LSEG workspace.

5.2. **A primitive trading algorithm.** In this section, I will derive a basic trading algorithm utilizing both HMM technology and the aforementioned Low Volatility Anomaly. The process of the algorithm is rather simple: for each day n,

(1) fit a k-state HMM on some subset of the preceding data. For example, one could fit a two-state 'run-reversal' model on the previous 100 days of trading data at each time step.
(2) Predict the next day's variance with the formula
$$\hat{\sigma}_{n+1} = \gamma(z_n)^T A_n \sigma_n$$
where $\gamma(z_n)$ is a $K \times 1$ vector of the responsibilities at time n, A_n is the $K \times K$ transition matrix at time n, and σ_n is a $K \times 1$ vector of the state standard deviations at time n.
(3) Check $\hat{\sigma}_{n+1}$ against some preset model tolerance t. If $\hat{\sigma}_{n+1} \leq t$, then the model will participate in the market. This means that it will either buy in if it is currently out of the market, or it will hold its position if it is currently in the market. If $\hat{\sigma}_{n+1} > t$, the model will not participate in the market, either selling its position or holding an out-of-market position.
(4) Observe the new period's return x_{n+1} and update the algorithm's price appropriately contingent upon the trading decision.

This loop then continues as new days are observed and added to the data. While this algorithm is quite simple, and we will see that it is effective, it is unfortunately quite computationally intensive and vulnerable to overfitting. This makes it far less suitable for high-frequency trading and unable to observe truly long-term trends in the market. To remedy this, we turn to online updating.

5.3. **Online trading algorithm.** I have devised a more complex trading algorithm using online HMM learning technology. Much like the primitive algorithm I discussed previously, this algorithm is based upon the Low Volatility

Anomaly, making its trading decisions by predicting the next day's variance given the expected current state, transition matrix, and state standard deviations. It begins by initializing the model parameters and relevant statistics (responsibilities, etc.) by fitting an HMM to the first n_{min} periods of the data. Then, it enters the core loop:

(1) Just like in the primitive algorithm, we use the formula
$$\hat{\sigma}_{n+1} = \gamma(z_n)^T A_n \sigma_n$$
to predict the next day's variance.
(2) Again, we check $\hat{\sigma}_{n+1}$ against some preset model tolerance t and trade based on this check.
(3) Observe the new period's return and update the price.
(4) In a novel step, we update the model's parameters in response to x_{n+1}.

The algorithm iterates through this loop as new data is passed in.

5.4. **Backtesting.** I tested both of my trading algorithms on the data I collected on the S&P 500. I chose (b_n) and t based on a grid search for their optimal values over the first 1500 days of data, then ran the algorithm on the remaining days. In this testing, I modeled two underlying states and separated the two based on their volatility. Thus, the model states can be indexed by l and h for the low and high volatility states, respectively. In this section, I will focus on the results from the online learning model, as this algorithm was faster and performed similarly well.

The results of the backtesting can be split into the bad news and the good news. First, the bad news - the modeled means and standard deviations fluctuate drastically over time, regularly fluctuating and spiking to high values over time. This can be seen in Figure 10. This figure shows plots of the

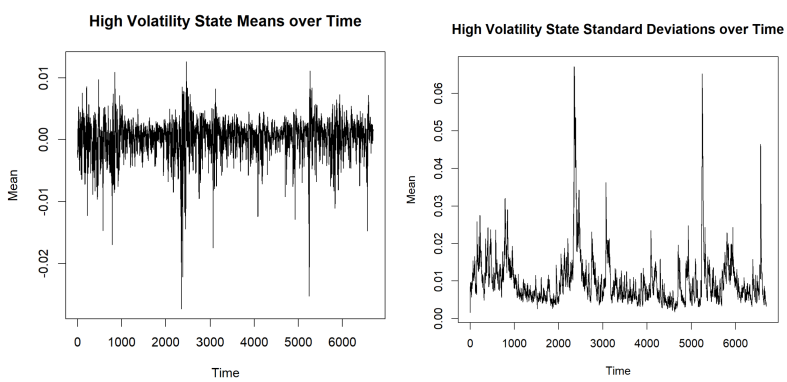

FIGURE 10. Violently fluctuating model parameters.

model's parameters over time, with clear fluctuations and peaks inconsistent

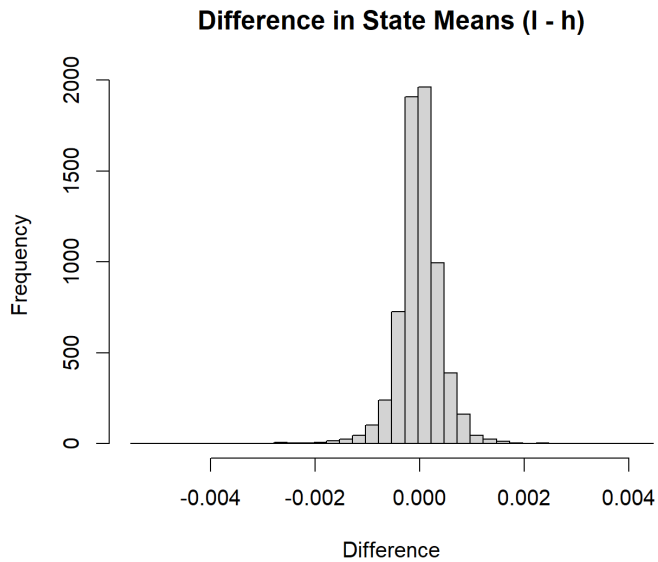

FIGURE 11. Histogram displaying the difference in state means throughout the backtest.

with the stability we would hope to see in a reliable model. Adding to this failure is the fact that the algorithm appears to have failed to properly distinguish between market regimes, and thus the parameters for both states fluctuate equally. This can be seen in Figures 11 and 12, which respectively display the difference in state means and standard deviations throughout the backtest. As becomes apparent from the figures, the means and standard deviations track quite closely with each other. The average difference in means is less than 0.0001 in absolute value, about 0.4% of the average low volatility mean. A similarly minuscule pattern of deviations can be observed in the standard deviations uncovered by the algorithm. A further way to verify the irrelevance of multiple states is to change the number of states the model uses and observe whether this changes the trading outcomes. However, when using 1 or 3 states, the trading outcomes are exactly the same as when using 2 states. It appears that the algorithm has entirely failed to distinguish different market regimes, and invites the question of whether more than one state is even necessary. This question arises as the variance is our key statistic and we allow it to fluctuate over time, thus modeling the current variance may not necessitate a difference in states at all.

One particularly noteworthy observation is that the primitive algorithm did not suffer from this failure. It was quite capable of distinguishing between

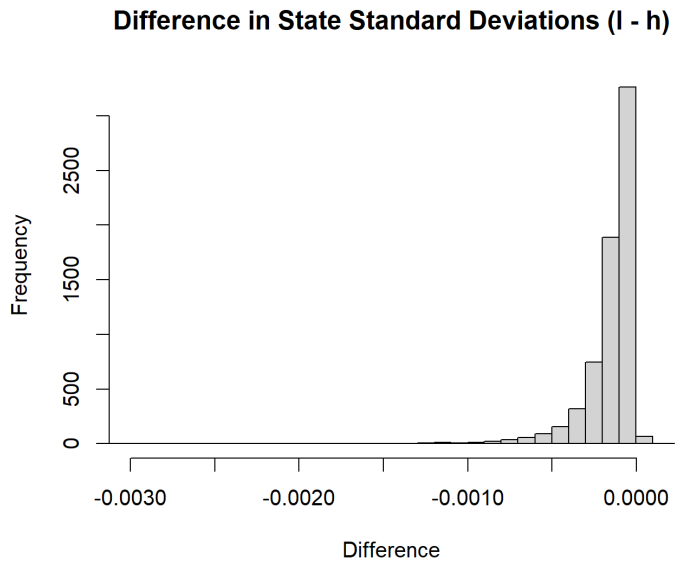

FIGURE 12. Histogram displaying the difference in state standard deviations throughout the backtest.

short-term runs and reversals. However, this did not translate to better performance. Unsurprisingly, the reason that the online algorithm performed better than the primitive algorithm is that, on days when the online algorithm chose to exit the market, the day's return was negative on average, while on days when the primitive algorithm chose to exit the market, the return was positive on average. This means that, on average, the online algorithm was avoiding negative returns, while the primitive algorithm avoided positive returns. This led to the online algorithm outperforming the buy and hold method, which in turn outperformed the primitive method. Thus, I am choosing to continue my analysis in the context of the much faster and more financially successful online algorithm.

Now for the good news. Figure 13 shows that the algorithm was able to significantly outperform a basic buy and hold strategy, earning about 35% more compared to the benchmark. This performance is rather strong, and reflects the ability of the algorithm to capture and exploit trends even when it is unable to distinguish market regimes. As can be seen in the figure, the algorithm is largely unable to avoid macro trends in the market, such as long-term reversals. One additional feature of note is that the model leaves the market quite sparingly. That is, it often stays in its optimistic mode, predicting a low variance for the next day. For the best-performing

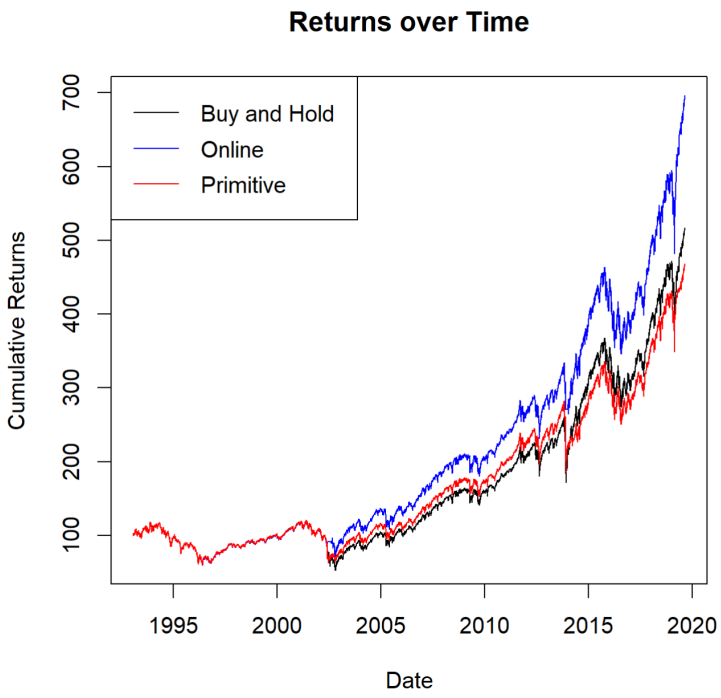

FIGURE 13. Cumulative returns of both of the algorithms and a basic 'buy and hold' strategy.

iterations of the algorithm, it agreed with the buy-and-hold strategy upwards of 98 percent of the time. However, by infrequently exploiting short-term trends, it was able to earn a larger return than would otherwise be seen in the market.

6. Conclusion

6.1. **Discussion of results.** This paper has demonstrated the capability of Hidden Markov Models to uncover and exploit trends in financial markets in order to turn a profit, particularly through online learning methods. Though the models utilized in this paper have failed to properly distinguish market regimes, they nonetheless produced returns exceeding the benchmark. This is especially notable as the models traded purely based on statistical reasoning, not utilizing any sort of news or other information that would normally influence a trading decision. The success of a purely statistical model in an environment dominated by news effects contradicts the hypothesis that market trends are random and non-exploitable. This carries implications for the stock market, both financially and economically. Most prominently, it implies

that markets do act irrationally at times, following patterns based on previous movements rather than a logical, more instantaneous price adjustment based on public information. However, it is important not to draw unequivocal causal conclusions from this success. It is possible that the success of the algorithms comes simply from the time needed by the market to adjust to price shocks, and that the models are able to identify when these adjustment periods begin and end. Nonetheless, the financial success of the model points towards the effectiveness of pure quantitative modeling in markets.

6.2. **Future opportunities.** There are various possible continuations of this research. First and foremost, it is most relevant to test the algorithm on other assets, in particular those that are less liquid or trade more cyclically than the S&P 500. Additionally, incorporating trading costs such as brokerage commissions could noticeably impact the profitability of the algorithm by increasing the difficulty of individual trades. However, the algorithm trades rather infrequently, so this is unlikely to be a major issue. Another possible dimension would be to consider a different period length, having the algorithm update every minute, every week, or on some other interval, rather than daily. This could also affect the profitability of the algorithm's performance significantly and presents an interesting opportunity to try to maximize returns. Finally, in this paper I have modeled financial data using multiple states due to my original focus on Markov regime switching. However, it seems likely that multiple states are not necessary in the online updating context. As such, it would be interesting to explore online methods which do not model multiple states, instead attempting to model the distribution of returns as a single probability distribution, then trading based on the variance of that distribution.

References

[1] Malcolm P. Baker, Brendan Bradley, and Jeffrey A. Wurgler. *Benchmarks as Limits to Arbitrage: Understanding the Low Volatility Anomaly*. SSRN Scholarly Paper 1585031. Social Science Research Network, Mar. 2010. DOI: 10.2139/ssrn.1585031.

[2] Christopher M. Bishop. *Pattern Recognition and Machine Learning*. en. Springer New York, 2006.

[3] Olivier Cappé. *Online EM Algorithm for Hidden Markov Models*. 2011. arXiv: 0908.2359 [stat.CO].

[4] C. Cuevas-Covarrubias, J. Iñigo-Martínez, and R. Jimenez-Padilla. "Gaussian mixtures and financial returns". In: *Discussiones Mathematicae: Probability and Statistics* 37 (2017), pp. 101–122. DOI: 10.7151/dmps.1190.

[5] José G. Dias, Jeroen K. Vermunt, and Sofia Ramos. "Clustering financial time series: New insights from an extended hidden Markov model". In: *European Journal of Operational Research* 243.3 (2015), pp. 852–864. DOI: https://doi.org/10.1016/j.ejor.2014.12.041.

[6] Stefano Ermon. *Learning in Latent Variable Models*.

[7] Gregory Gundersen. *Expectation-Maximization*. gregorygundersen.com. Nov. 2019. URL: https://gregorygundersen.com/blog/2019/11/10/em/.

[8] Gregory Gundersen. *Inference for Hidden Markov Models*. gregorygundersen.com. Nov. 2020. URL: https://gregorygundersen.com/blog/2020/11/28/hmms/.

[9] Chandrashekar Lakshminarayanan and Csaba Szepesvári. *Linear Stochastic Approximation: Constant Step-Size and Iterate Averaging*. 2017. arXiv: 1709.04073 [cs.LG].

[10] Bo Li. "An explanation for the distribution characteristics of stock returns". In: *arXiv preprint* (Dec. 2023). arXiv: 2312.02472v1 [q-fin.ST].

[11] John M. Maheu and Thomas H. McCurdy. "Identifying Bull and Bear Markets in Stock Returns". In: *Journal of Business & Economic Statistics* 18.1 (2000), pp. 100–112. DOI: 10.1080/07350015.2000.10524851. eprint: https://www.tandfonline.com/doi/pdf/10.1080/07350015.2000.10524851.

[12] Lennart Oelschläger and Timo Adam. "Detecting bearish and bullish markets in financial time series using hierarchical hidden Markov models". In: *Statistical Modelling* 23.2 (2023), pp. 107–126. DOI: 10.1177/1471082X211034048. eprint: https://doi.org/10.1177/1471082X211034048.

[13] Ingmar Visser and Maarten Speekenbrink. "depmixS4: An R Package for Hidden Markov Models". In: *Journal of Statistical Software* 36.7 (2010), pp. 1–21.

[14] Markus Vogl. "Quantitative modelling frontiers: a literature review on the evolution in financial and risk modelling after the financial crisis (2008–2019)". In: *SN Business & Economics* 2.12 (2022), p. 183. DOI: 10.1007/s43546-022-00359-3.

[15] Shaul Wisebourt. "Hierarchical hidden Markov model of high-frequency market regimes using trade price and limit order book information". Master's Thesis. University of Waterloo, 2011.

Appendix A.
Summation Notation of the Log-Likelihood Function

We have the likelihood function

$$Q(\theta, \theta^{old}) = \sum_Z p(Z|X, \theta^{old}) \ln p(X, Z|\theta)$$

Here, Z denotes the set of all possible latent state realizations $\{z_1, \ldots, z_n\}$; that is, we are summing over all possible values of the K-long vector in which one entry is one and the rest are zero. Using equation (13.10) for $p(X, Z|\theta)$ in Bishop, as well as basic log rules, see that

$$\begin{aligned}
Q(\theta, \theta^{old}) &= \sum_Z p(Z|X, \theta^{old}) \ln \left(p(z_1|\pi) \left[\prod_{n=2}^{N} p(z_n|z_{n-1}, A) \right] \prod_{m=1}^{N} p(x_m|z_m, \phi) \right) \\
&= \sum_Z p(Z|X, \theta^{old}) \left(\ln p(z_1|\pi) + \sum_{n=2}^{N} \ln p(z_n|z_{n-1}, A) + \sum_{m=1}^{N} \ln p(x_m|z_m, \phi) \right) \\
&= \mathbb{E}_{p(Z|X,\theta^{old})} \left[\ln p(z_1|\pi) \right] + \sum_{n=2}^{N} \left(\mathbb{E}_{p(Z|X,\theta^{old})} \left[\ln p(z_n|z_{n-1}, A) \right] \right) \\
&\quad + \sum_{m=1}^{N} \left(\mathbb{E}_{p(Z|X,\theta^{old})} \left[\ln p(x_m|z_m, \phi) \right] \right) \\
&= \mathbb{E} \left[\ln \pi_k \right] + \sum_{n=2}^{N} \left(\mathbb{E} \left[\ln \left(\prod_{k=1}^{K} \prod_{j=1}^{K} A_{jk}^{z_{n-1,j} z_{nk}} \right) \right] \right) \\
&\quad + \sum_{m=1}^{N} \left(\mathbb{E} \left[\ln \prod_{k=1}^{K} p(x_m|z_m, \phi)^{z_{mk}} \right] \right) \\
&= \sum_{k=1}^{K} \left(\mathbb{E} \left[z_{nk} \right] \ln \pi_k \right) + \sum_{n=2}^{N} \sum_{k=1}^{K} \sum_{j=1}^{K} \left(\mathbb{E} \left[z_{n-1,j} z_{nk} \right] \ln A_{jk} \right) \\
&\quad + \sum_{m=1}^{N} \sum_{k=1}^{K} \left(\mathbb{E} \left[z_{mk} \ln p(x_m|\phi_k) \right] \right)
\end{aligned}$$

We can express this in terms of our $\gamma(z_n)$ and $\xi(z_{n-1}, z_n)$, finding that

$$\begin{aligned}
Q(\theta, \theta^{old}) = &\sum_{k=1}^{K} \gamma(z_{1k}) \ln \pi_k + \sum_{n=2}^{N} \sum_{j=1}^{K} \sum_{k=1}^{K} \xi(z_{n-1,j}, z_{nk}) \ln A_{jk} \\
&+ \sum_{m=1}^{N} \sum_{k=1}^{K} \gamma(z_{mk}) \ln p(x_m|\phi_k)
\end{aligned} \tag{21}$$